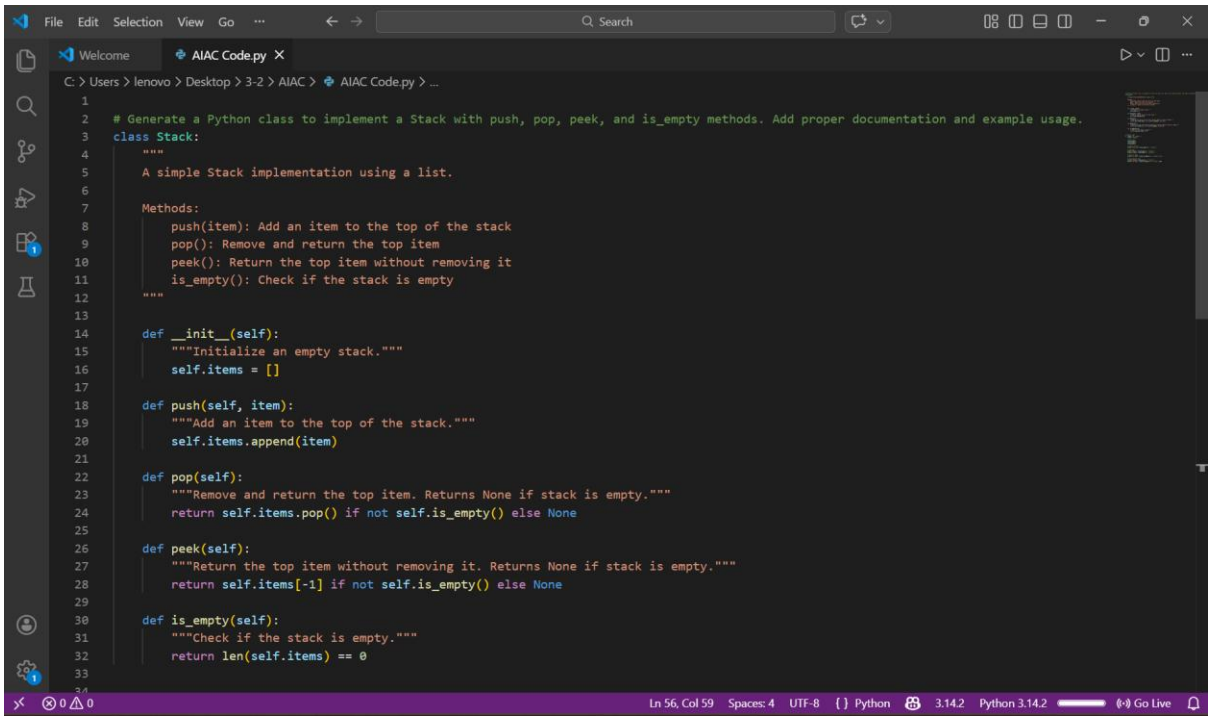
N. Shree Deepthi Batch – 37

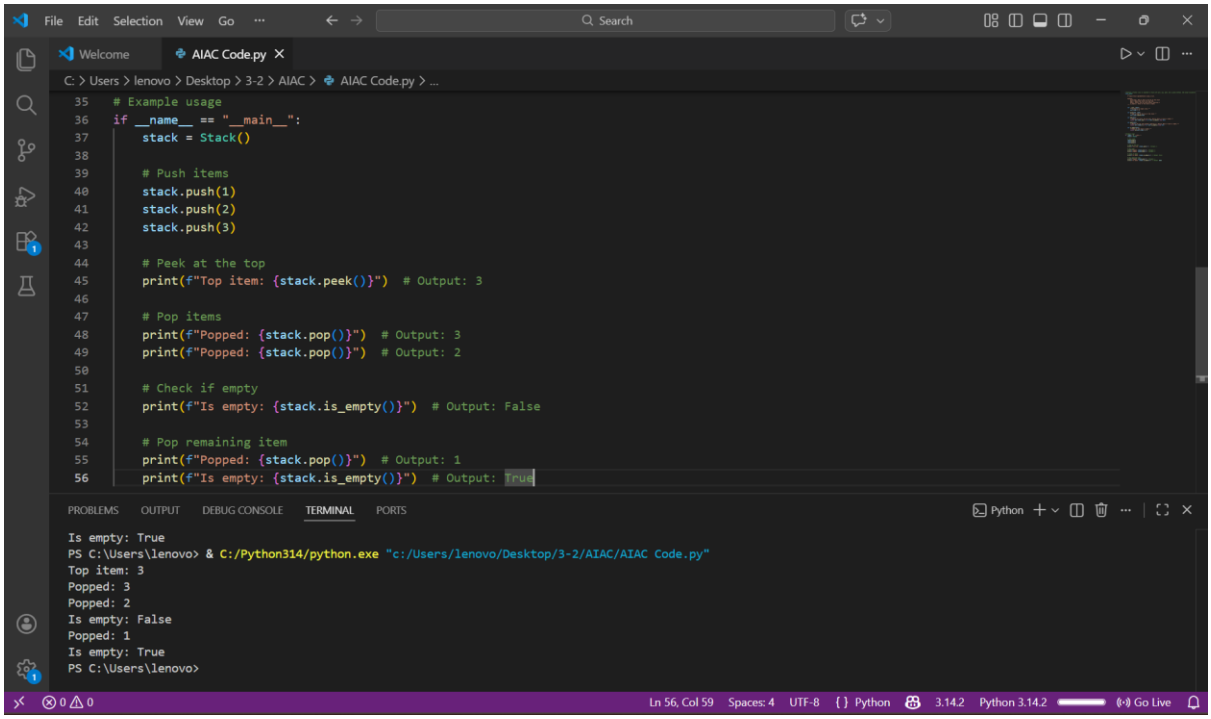
AI-ASSISTED CODING ASSIGNMENT 11

Task 1 – Stack Using AI Guidance

AI Prompt Used

"Generate a Python class to implement a Stack with push, pop, peek, and is_empty methods. Add proper documentation and example usage."





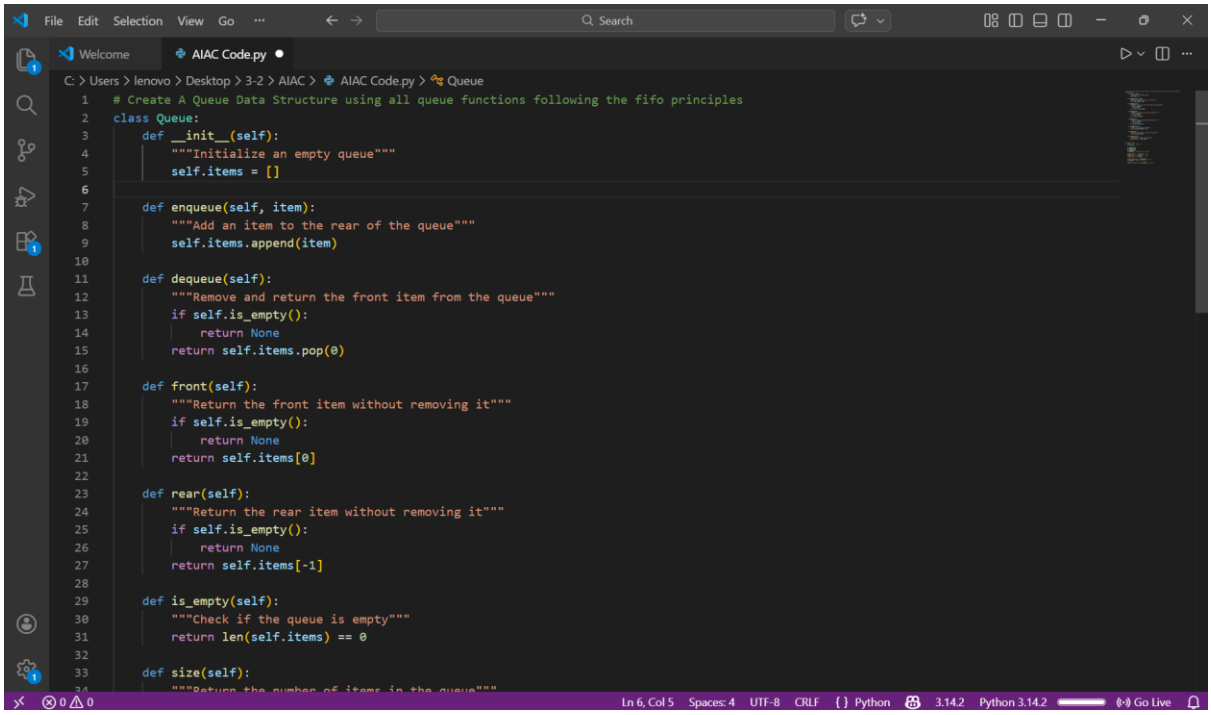
Explanation

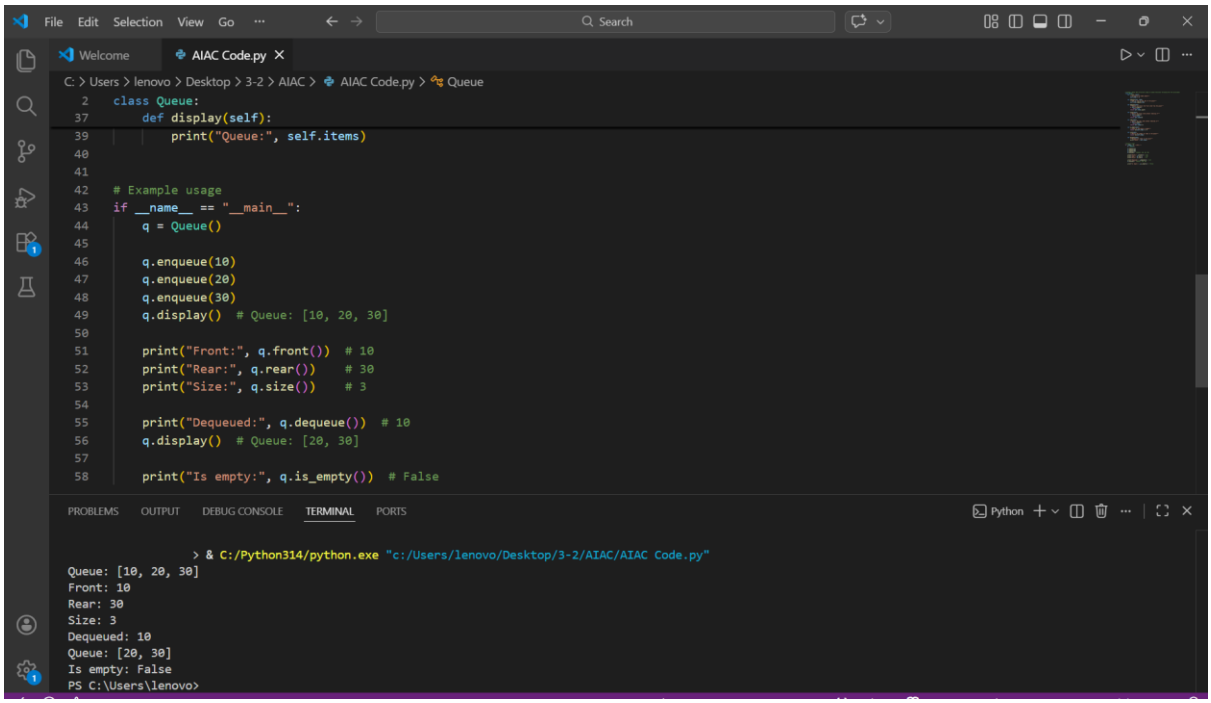
- **Stack follows LIFO principle**
- **Implemented using Python list**
- **append() → push**
- **pop() → remove last element**
- **Time Complexity:**
  - **Push: O(1)**
  - **Pop: O(1)**

Task 2 – Queue Design

AI Prompt Used

"Create a Queue class in Python that follows FIFO principle and includes all queue functions."





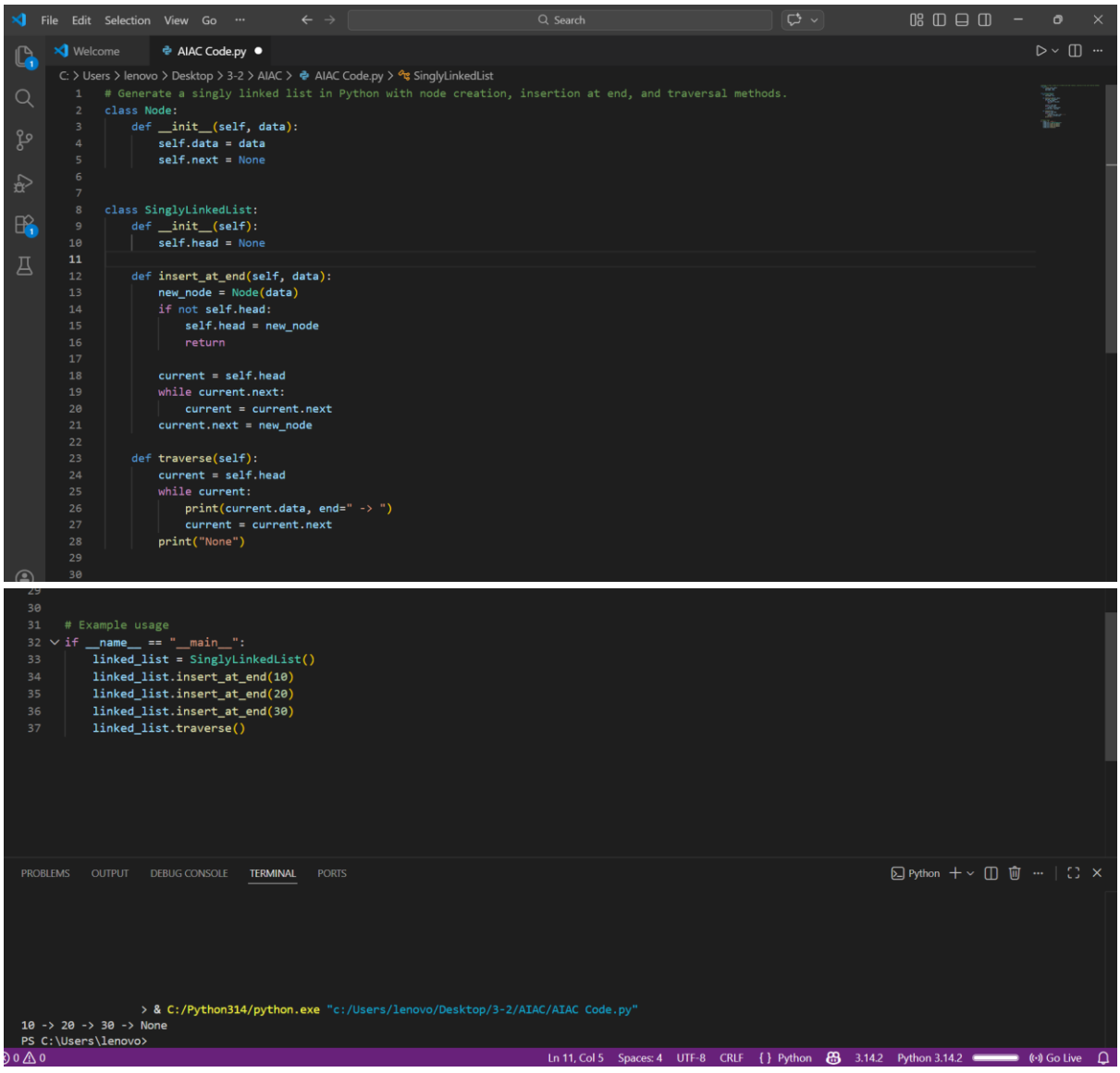
Explanation

- **Queue follows FIFO principle**
- **Insert at end → append()**
- **Remove from front → pop(0)**
- **Note: pop(0) is O(n).**
  For optimization, collections.deque is recommended.

Task 3 – Singly Linked List

AI Prompt Used

"Generate a singly linked list in Python with node creation, insertion at end, and traversal methods."



```
# Generate a singly linked list in Python with node creation, insertion at end, and traversal methods.
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


class SinglyLinkedList:
    def __init__(self):
        self.head = None

    def insert_at_end(self, data):
        new_node = Node(data)
        if not self.head:
            self.head = new_node
            return

        current = self.head
        while current.next:
            current = current.next
        current.next = new_node

    def traverse(self):
        current = self.head
        while current:
            print(current.data, end=" -> ")
            current = current.next
        print("None")


# Example usage
if __name__ == "__main__":
    linked_list = SinglyLinkedList()
    linked_list.insert_at_end(10)
    linked_list.insert_at_end(20)
    linked_list.insert_at_end(30)
    linked_list.traverse()
```

```
> & C:/Python314/python.exe "c:/Users/lenovo/Desktop/3-2/AIAC/AIAC Code.py"
10 -> 20 -> 30 -> None
PS C:\Users\lenovo>
```

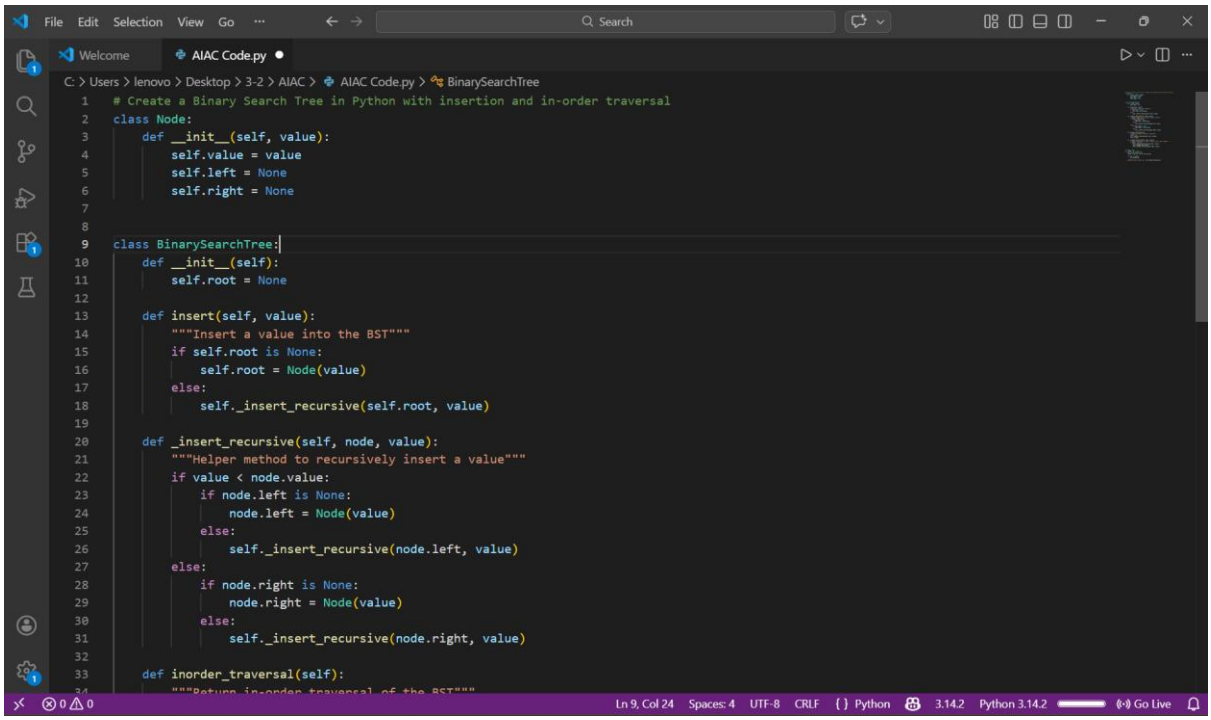
Explanation

- **Each node contains:**
  - **Data**
  - **Pointer to next node**
- **Insertion at end → O(n)**
- **Traversal → O(n)**

Task 4 – Binary Search Tree

AI Prompt Used

"Create a Binary Search Tree in Python with insertion and in-order traversal."



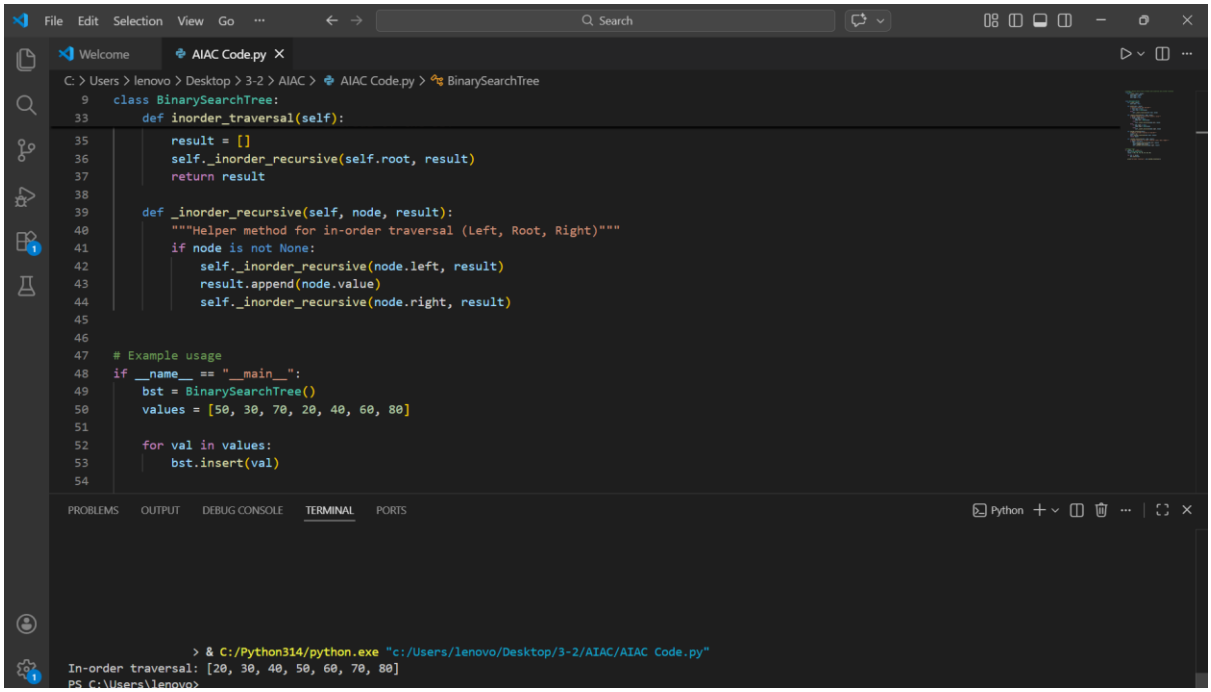
```python
# Create a Binary Search Tree in Python with insertion and in-order traversal
class Node:
    def __init__(self, value):
        self.value = value
        self.left = None
        self.right = None


class BinarySearchTree:
    def __init__(self):
        self.root = None

    def insert(self, value):
        """Insert a value into the BST"""
        if self.root is None:
            self.root = Node(value)
        else:
            self._insert_recursive(self.root, value)

    def _insert_recursive(self, node, value):
        """Helper method to recursively insert a value"""
        if value < node.value:
            if node.left is None:
                node.left = Node(value)
            else:
                self._insert_recursive(node.left, value)
        else:
            if node.right is None:
                node.right = Node(value)
            else:
                self._insert_recursive(node.right, value)

    def inorder_traversal(self):
        """Return in-order traversal of the BST"""
```



```python
class BinarySearchTree:
    def inorder_traversal(self):
        result = []
        self._inorder_recursive(self.root, result)
        return result

    def _inorder_recursive(self, node, result):
        """Helper method for in-order traversal (Left, Root, Right)"""
        if node is not None:
            self._inorder_recursive(node.left, result)
            result.append(node.value)
            self._inorder_recursive(node.right, result)


# Example usage
if __name__ == "__main__":
    bst = BinarySearchTree()
    values = [50, 30, 70, 20, 40, 60, 80]

    for val in values:
        bst.insert(val)
```

```
> & C:/Python314/python.exe "c:/Users/lenovo/Desktop/3-2/AIAC/AIAC Code.py"
In-order traversal: [20, 30, 40, 50, 60, 70, 80]
PS C:\Users\lenovo>
```

Explanation

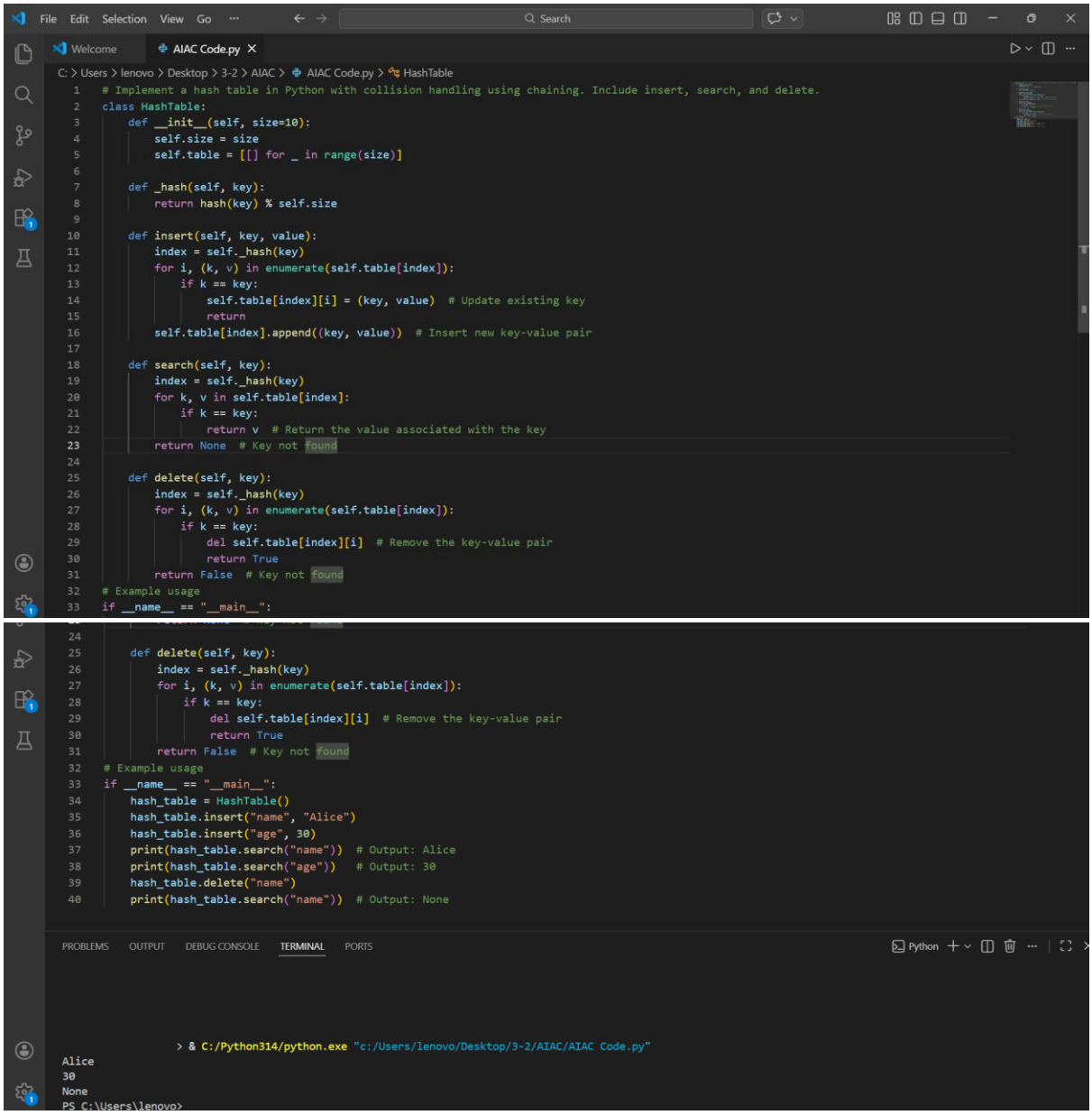
- **Left child < Root**

- **Right child > Root**
- **In-order traversal gives sorted output**
- **Average insertion time: O(log n)**

Task 5 – Hash Table (Using Chaining)

AI Prompt Used

"Implement a hash table in Python with collision handling using chaining. Include insert, search, and delete."



```python
# Implement a hash table in Python with collision handling using chaining. Include insert, search, and delete.
class HashTable:
    def __init__(self, size=10):
        self.size = size
        self.table = [[] for _ in range(size)]

    def _hash(self, key):
        return hash(key) % self.size

    def insert(self, key, value):
        index = self._hash(key)
        for i, (k, v) in enumerate(self.table[index]):
            if k == key:
                self.table[index][i] = (key, value)  # Update existing key
                return
        self.table[index].append((key, value))  # Insert new key-value pair

    def search(self, key):
        index = self._hash(key)
        for k, v in self.table[index]:
            if k == key:
                return v  # Return the value associated with the key
        return None  # Key not found

    def delete(self, key):
        index = self._hash(key)
        for i, (k, v) in enumerate(self.table[index]):
            if k == key:
                del self.table[index][i]  # Remove the key-value pair
                return True
        return False  # Key not found
# Example usage
if __name__ == "__main__":
    hash_table = HashTable()
    hash_table.insert("name", "Alice")
    hash_table.insert("age", 30)
    print(hash_table.search("name"))  # Output: Alice
    print(hash_table.search("age"))   # Output: 30
    hash_table.delete("name")
    print(hash_table.search("name"))  # Output: None
```

```
> & C:/Python314/python.exe "c:/Users/lenovo/Desktop/3-2/AIAC/AIAC Code.py"
Alice
30
None
PS C:\Users\lenovo>
```

Explanation

- **Hash function maps key → index**
- **Collision handled using chaining (list of tuples)**
- **Average time complexity:**
    - **Insert: O(1)**
    - **Search: O(1)**
    - **Delete: O(1)**

Conclusion

Using AI assistance:

- **Improve speed of execution**
- **Added complexity analysis**
- **Enhanced readability of code**
- **Learned from optimization suggestions**